Student notes — Session 2 · printf formatting, scanf, arithmetic and math functions

Slides: [Week_01_Programming_Principles_I_C.pdf](../slides/Week_01_Programming_Principles_I_C.pdf). These notes are the notes pane of the student copy of the deck.

Slide 10 — Standard output: printf

How to try the examples in these notes. Every example below is a small program. To try one:

1. Open Visual Studio Code.
2. Click **File**, then **Open Folder**, and open your `c_week1` folder.
3. Click **File**, then **New Text File**. Type the program. Click **File**, then **Save As**, and give it a name ending in `.c`, for example `test.c`.
4. Click **View**, then **Terminal**. A window opens at the bottom. That is the terminal.
5. In the terminal, type `gcc -Wall test.c -o test` and press Enter. If nothing is printed, good. That means it worked.
6. Type `./test` and press Enter. (In Windows PowerShell, type `.\test` instead.) Your program runs.

Now the slide.

What printf does. `printf` puts words and numbers on the screen. Without it, your program is silent.

Example 1. Print fixed words.

```
printf("Hello!\n");
```

The screen shows:

```
Hello!
```

Whatever is between the quotes goes on the screen. The `\n` at the end is not shown. It means "go to the next line".

Example 2. Print a number that is in a variable.

```
int age = 20;
printf("Age: %d\n", age);
```

The screen shows:

```
Age: 20
```

Look at the `printf` line. It has two parts, with a comma between them:

- Part 1, before the comma: the words in quotes, `"Age: %d\n"`.
- Part 2, after the comma: the variable, `age`.

Inside the words there is `%d`. Think of `%d` as a **hole**. When the program runs, the value of `age`, which is 20, is put into the hole. So `%d` is not shown. The number 20 is shown in its place.

The hole must match the type of the variable. There is a different hole for each type.

Variable type	Hole to use	Example	Screen shows
`int` (whole number)	`%d`	`int n = 7;` `printf("%d\n", n);`	`7`
`double` (number with decimal point)	`%f`	`double p = 3.75;` `printf("%f\n", p);`	`3.750000`
`double`, only 2 digits after the point	`%.2f`	`double p = 3.75;` `printf("%.2f\n", p);`	`3.75`
`double`, no digits after the point	`%.0f`	`double p = 3.75;` `printf("%.0f\n", p);`	`4`
`char` (one letter)	`%c`	`char g = 'A';` `printf("%c\n", g);`	`A`

Look at the three `double` rows. Same variable, same value 3.75, three different holes, three different results. Plain `%f` always shows six digits after the point. `%.2f` shows two. `%.0f` shows none and rounds 3.75 up to 4. The number after the dot tells `printf` **how many digits after the decimal point to show**. The variable itself does not change.

For money, always use `%.2f`. Prices have two digits after the point.

Example 3. Wrong hole. What if you use `%d` for a `double`?

```
double p = 3.75;
printf("%d\n", p);
```

The screen shows something like `0` or `1074266112`. Nonsense. The hole and the variable do not match. The computer does not stop you. It just prints rubbish.

To catch this mistake, ask the compiler to check for you. From now on, always compile like this:

```
gcc -Wall test.c -o test
```

`-Wall` means "warn me about all problems". With `-Wall`, the compiler prints a warning for Example 3. It says the format expects an `int` but you gave it a `double`. Read the warning, fix the hole, compile again.

Rule: when your output looks like nonsense, check your `%` letters first.

Example 4. Two holes in one line.

```
int a = 3;
int b = 4;
printf("%d plus %d\n", a, b);
```

The screen shows:

```
3 plus 4
```

Two holes, so two variables after the comma. The first variable goes into the first hole. The second goes into the second. Count them: the number of holes and the number of variables must be the same.

Example 5. Words and a number together.

```
double price = 12.5;
printf("The price is %.2f manat\n", price);
```

The screen shows:

```
The price is 12.50 manat
```

Example 6. What `\n` does. Two `printf` lines, without `\n`:

```
printf("one");
printf("two");
```

The screen shows:

```
onetwo
```

Both words on one line, stuck together. Now with `\n`:

```
printf("one\n");
printf("two\n");
```

The screen shows:

```
one
two
```

So `\n` means "the next thing starts on a new line".

Good habit: always print a label. Do not print just `20`. Print `Age: 20`. When your program prints five bare numbers and one is wrong, you cannot tell which is which.

Try it yourself.

1. Make a variable `int year = 2026;` and print `Year: 2026`.
2. Make a variable `double height = 1.8;` and print `Height: 1.80`.
3. Print your name and age on one line, like `Ali is 19 years old`, with the age coming from an `int` variable.

Slide 11 — Standard input: scanf

`printf` sends things **out** to the screen. `scanf` brings things **in** from the keyboard. With `scanf`, your program can ask you a question and use your answer.

Example 1. Ask for a number.

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Enter age: ");
    scanf("%d", &age);
    printf("You are %d years old\n", age);
    return 0;
}
```

Compile and run it. This is what happens:

1. The screen shows `Enter age:` and then **nothing**. The program is waiting. The blinking cursor is right after the colon.
2. You type `19` and press Enter.
3. The screen shows `You are 19 years old`.

So `scanf` does three things. It waits for you to type. It reads what you typed. It stores it in the variable `age`.

Look closely at the scanf line. `scanf("%d", &age);`

- `%d` is the hole, like in `printf`. It says "I expect a whole number".
- `&age` is the variable, but with `&` in front of it. **The `&` is not a mistake. You must write it.**

Why `&`? Think of the variable `age` as a box. `printf` only needs to **look inside** the box, so you give it the box's contents. `scanf` needs to **put something into** the box. To do that it needs to know **where the box is**. `&age` means "the place where `age` is". Without `&`, `scanf` does not know where to put the number.

If you forget the `&`: the program still compiles. But when you run it, it crashes, or `age` stays empty and you see a nonsense number. With `-Wall`, the compiler warns you: `format '%d' expects argument of type 'int *'`. When you see that warning, you forgot `&`.

Example 2. Ask for a number with a decimal point.

```
double height;
printf("Enter height: ");
scanf("%lf", &height);
printf("Height: %.2f\n", height);
```

You type `1.82` and press Enter. The screen shows `Height: 1.82`.

Careful: for a `double`, `scanf` uses `%lf`, not `%f`. That is an L, not a 1. This is just how C is. Remember it like this:

Type	printf hole	scanf hole
`int`	`%d`	`%d`
`double`	`%f` or `%.2f`	`%lf`

Example 3. Ask for two numbers, one after the other. This is the program on the slide.

```
int age;
double height;
printf("Enter age: ");
scanf("%d", &age);
printf("Enter height: ");
scanf("%lf", &height);
printf("Age %d, height %.2f\n", age, height);
```

Run it. Type `19`, press Enter. Type `1.82`, press Enter. The screen shows `Age 19, height 1.82`. So `age` holds 19 and `height` holds 1.82.

You may also type both on one line, `19 1.82`, and press Enter once. `scanf` skips the space and takes the next number. Both ways work.

Example 4. Two numbers in one scanf.

```
int a, b;
printf("Two numbers: ");
scanf("%d %d", &a, &b);
printf("Sum = %d\n", a + b);
```

You type `4 9` and press Enter. The screen shows `Sum = 13`. Two holes, so two variables, each with `&`.

Why no `\n` in the question? `printf("Enter age: ")` has no `\n`. So the cursor stays on the same line, and you type right after the colon. That looks like a proper question. If you put `\n`, you would type on the next line. Both work. No `\n` looks nicer.

What if you type a letter instead of a number? The program gets confused and `age` gets no value. In week 7 you learn how to check for that. For now, type numbers.

Try it yourself.

1. Write a program that asks "Enter a number: " and then prints `You typed 5` (or whatever you typed).
2. Change it so it asks for a price with a decimal point and prints `Price: 12.50`.
3. Ask for two whole numbers and print them the other way round: type `3 8`, see `8 3`.

Slide 12 — Expressions and operators

An **expression** is a small calculation, like `a + b`. C has five basic operators:

Operator	Meaning	Example	Result
`+`	add	`7 + 3`	`10`
`-`	subtract	`7 - 3`	`4`
`*`	multiply (this is the star, not x)	`7 * 3`	`21`
`/`	divide	`7 / 3`	`2` (yes, 2. Read on.)
`%`	remainder after dividing	`7 % 3`	`1`

You can print a calculation directly. You do not need to store it first:

```
int a = 7;
int b = 3;
printf("%d\n", a + b);
```

The screen shows `10`.

Surprise 1: dividing whole numbers gives a whole number. `7 / 3` is 2, not 2.33. Why? Both 7 and 3 are `int`. When C divides an `int` by an `int`, the answer is an `int`. The part after the decimal point is **thrown away**. Not rounded. Thrown away.

More examples:

Expression	Result	Why
`7 / 2`	`3`	3.5, and the .5 is thrown away
`9 / 2`	`4`	4.5, and the .5 is thrown away
`10 / 5`	`2`	exact, nothing to throw away
`1 / 2`	`0`	0.5, and the .5 is thrown away

If you want the decimal part, at least one of the numbers must be a `double`:

Expression	Result
`7 / 2`	`3`
`7.0 / 2`	`3.5`
`7 / 2.0`	`3.5`

Rule for now: **if you want decimals, use `double`.**

Surprise 2: `%` is the remainder. It is called "modulo". `7 % 3` asks: divide 7 by 3, what is left over? 3 goes into 7 two times, and 1 is left over. So `7 % 3` is 1.

More examples:

Expression	Result	Why
`10 % 3`	`1`	3 × 3 = 9, and 1 is left
`10 % 5`	`0`	5 × 2 = 10, nothing left
`8 % 2`	`0`	even numbers leave 0 when divided by 2
`9 % 2`	`1`	odd numbers leave 1
`123 % 10`	`3`	the last digit

`%` only works with whole numbers.

Surprise 3: multiplication and division come before addition and subtraction. Exactly as in school.

Expression	Result	Why
`2 + 3 * 4`	`14`	3 × 4 first, that is 12, then 2 + 12
`(2 + 3) * 4`	`20`	brackets first: 2 + 3 is 5, then 5 × 4
`10 - 4 / 2`	`8`	4 / 2 first, that is 2, then 10 - 2

If you want the addition first, use brackets.

The four lines on the slide. With `a = 7` and `b = 3`:

```
a + b      → 10
a / b      → 2
a % b      → 1
2 + 3 * 4  → 14
```

Try it yourself. Before you run each one, write down what you think it prints. Then run it and check.

1. `printf("%d\n", 15 / 4);`
2. `printf("%d\n", 15 % 4);`
3. `printf("%d\n", 1 + 2 * 3);`
4. `printf("%f\n", 15.0 / 4);`

(Answers: 3, 3, 7, 3.750000.)

Slide 13 — Math functions

Adding and multiplying are built into C. Square root is not. To use square root, powers and rounding, you need the **math library**. You get it with one extra line at the top of your program:

```
#include <stdio.h>
#include <math.h>
```

Now you have two `#include` lines. The first one gives you `printf` and `scanf`. The second one gives you the math functions. Without the second line, `sqrt` does not exist and the compiler complains.

How a function works. You write the name, then brackets, then what you want inside the brackets. The answer comes back. You can print it or store it in a variable.

```
double x = 16.0;
printf("%.2f\n", sqrt(x));
```

The screen shows `4.00`. The function `sqrt` took 16 and gave back 4.

The five functions.

Function	What it does	Example	Gives
`sqrt(x)`	square root	`sqrt(16.0)`	`4`

Function	What it does	Example	Gives
`pow(a, b)`	a to the power b	`pow(2, 5)`	`32` (2×2×2×2×2)
`round(x)`	nearest whole number	`round(3.6)`	`4`
`ceil(x)`	round up	`ceil(3.2)`	`4`
`floor(x)`	round down	`floor(3.8)`	`3`

Notice `pow` needs two things inside the brackets, with a comma between them.

All of them give back a `double`. Even `round`. So print them with `%f` or `%.2f` or `%.0f`, never with `%d`. This is why the slide prints `round(3.6)` with `%.0f` and gets `4`.

Example: the full program on the slide.

```
#include <stdio.h>
#include <math.h>
int main(void) {
    double x = 16.0;
    printf("sqrt = %.2f\n", sqrt(x));
    printf("power = %.2f\n", pow(2, 5));
    printf("round = %.0f\n", round(3.6));
    return 0;
}
```

The screen shows:

```
sqrt = 4.00
power = 32.00
round = 4
```

Storing the answer in a variable.

```
double side = 9.0;
double root = sqrt(side);
printf("Root: %.1f\n", root);
```

The screen shows `Root: 3.0`.

Compile note for Linux and WSL. On Linux, and in WSL on Windows, you must add `-lm` at the end of the compile command:

```
gcc -Wall test.c -o test -lm
```

If you forget it, you see the error `undefined reference to 'sqrt'`. That error means: add `-lm`. On Windows with the course installer, and on a Mac, you do not need `-lm`. Adding it anyway does no harm.

The question on the slide. A rectangle has width `w` and height `h`. The diagonal is the square root of $w^2 + h^2$. In C:

```
double d = sqrt(w * w + h * h);
```

Or with `pow`: `sqrt(pow(w, 2) + pow(h, 2))`. Both are right. The first is simpler.

Try it yourself.

1. Print the square root of 81. (Expect `9.00`.)
2. Print 3 to the power 4. (Expect `81.00`.)

3. Print `ceil(2.1)` and `floor(2.9)` with `%.0f`. (Expect `3` and `2`.)

Slide 14 — Worked example: circle calculator

This program uses everything from this week. Read it slowly. Every line is something you already know.

Step 1: the plan, before any code. The problem: read the radius of a circle, print its area and circumference. The plan:

1. Ask for the radius. Call it `r`.
2. Area = π × r × r.
3. Circumference = 2 × π × r.
4. Print both, with two digits after the point.

There is no C in the plan. Always write the plan first.

Step 2: the code.

```
#include <stdio.h>
int main(void) {
    double r, area, c;
    const double PI = 3.14159;
    printf("Radius: ");
    scanf("%lf", &r);
    area = PI * r * r;
    c = 2 * PI * r;
    printf("Area: %.2f\n", area);
    printf("Circumference: %.2f\n", c);
    return 0;
}
```

Line by line.

- `#include <stdio.h>` because we use `printf` and `scanf`. No `math.h` here. There is no square root in this program.
- `double r, area, c;` makes three `double` variables in one line. The commas separate the names. Same as writing three lines: `double r; double area; double c;`.
- `const double PI = 3.14159;` makes a variable called `PI` with 3.14159 in it. The word `const` means "constant": this value is locked and cannot be changed later. Capital letters are the custom for constants. Why not just write 3.14159 twice? Because `PI` is easier to read, and if you want more digits later, you change one line.
- `printf("Radius: ");` asks the question. No `\n`, so you type on the same line.
- `scanf("%lf", &r);` reads a `double`, so `%lf`. Into `r`, so `&r`.
- `area = PI * r * r;` calculates the right side, then stores the answer in `area`. Plan step 2.
- `c = 2 * PI * r;` plan step 3.
- The two `printf` lines print with a label and two decimals. Plan step 4.
- `return 0;` says "finished, all fine".

Step 3: run it. Save as `circle.c`. In the terminal:

```
gcc -Wall circle.c -o circle
./circle
```

The screen shows `Radius: `. Type `3` and press Enter. The screen shows:

```
Area: 28.27
Circumference: 18.85
```

Try radius `1`. You get `Area: 3.14` and `Circumference: 6.28`. Try `2.5`. You get `Area: 19.63` and `Circumference: 15.71`.

Extension: print the diameter too. The diameter is 2 × r. Add one line before `return 0;`:

```
printf("Diameter: %.2f\n", 2 * r);
```

For radius 3 it prints `Diameter: 6.00`. Here the calculation `2 * r` is put straight into the hole. You could also make a new variable, `double d = 2 * r;`, and print `d`. Both work.

If your output is wrong, check these three things first.

1. Did you write `&r` in `scanf`, with the `&`?
2. Did you write `%lf` in `scanf`, with the L?
3. Is there a semicolon at the end of every line inside `main`?

Slide 15 — Debugging mindset

Every program has mistakes in it at first. Every program, every programmer, every day. That is normal. A mistake in a program is called a **bug**. Finding and fixing bugs is called **debugging**.

Changing random things until it works is **not** debugging. It is guessing.

The program on the slide has two bugs. Here it is:

```
#include <stdio.h>
int main(void) {
    int a, b;
    printf("Two numbers: ");
    scanf("%d %d", a, b);
    printf("Sum = %d\n", a + b)
    return 0;
}
```

- **Bug 1: a missing semicolon.** The line `printf("Sum = %d\n", a + b)` has no `;` at the end. The compiler stops with an **error**. It prints something like `error: expected ';' before 'return'` and the line number. No program is made until you fix it.
- **Bug 2: missing `&`.** The line `scanf("%d %d", a, b)` should be `scanf("%d %d", &a, &b)`. The compiler does **not** stop for this. It makes a program anyway. With `-Wall` it prints a **warning**. If you run the program, it crashes or prints a wrong sum.

The difference between an error and a warning.

	Error	**Warning**
Does the compiler make a program?	No	Yes
Must you fix it?	Yes, or nothing runs	Yes, or it runs wrong
Do you see it without `-Wall`?	Yes	Often not

How to fix a program with several problems.

1. Compile with `-Wall`.

2. Read the **first** error only. Ignore the rest for now. Later errors are often caused by the first one.
3. Fix that one thing.
4. Compile again. Repeat.
5. When there are no errors left, read the warnings and fix them the same way.

The fixed program:

```
#include <stdio.h>
int main(void) {
    int a, b;
    printf("Two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Sum = %d\n", a + b);
    return 0;
}
```

Run it, type `4 9`, press Enter. The screen shows `Sum = 13`.

Try it yourself. Type the broken version exactly as on the slide. Compile it with `-Wall`. Read the error. Fix only the semicolon. Compile again. Read the warning. Fix the `&`. Compile again. Run with 4 and 9.

Slide 16 — Quick checks

Answer these yourself first. Then read the answers.

Q1. What is the difference between source code and an executable?

Source code is the text you write, in the file `hello.c`. You can read it. The computer cannot run it. The executable is the new file the compiler makes from it, for example `hello` or `hello.exe`. The computer can run that one. You cannot read it.

Q2. What does `#include <stdio.h>` give us?

`printf` and `scanf`. Without that line, the compiler does not know what `printf` is.

Q3. Why does `scanf` need `&` before a variable?

Because `scanf` puts a value **into** the variable. To do that it needs to know **where** the variable is. `&` gives it the place.

Q4. What is printed by `int x = 4; x = x + 3; printf("%d", x);`?

`7`. First `x` is 4. Then the right side is calculated: 4 + 3 is 7. Then 7 is stored in `x`. Then 7 is printed.

Q5. Why is `7 / 2` equal to 3 in C?

Both 7 and 2 are whole numbers (`int`). Dividing an `int` by an `int` gives an `int`. The .5 is thrown away. To get 3.5, write `7.0 / 2`.

Q6. Write one line that prints a `double` with 2 decimal places.

```
printf("%.2f\n", x);
```

Slide 17 — Practice lab: three small programs

Write these three programs at home. Each one is the circle calculator with a different formula. Use the same shape: ask, calculate, print with a label.

1. Temperature converter. Ask for a temperature in Celsius. Print it in Fahrenheit. The formula is F = C × 9 / 5 + 32.

Example run: you type `25`, the screen shows `Fahrenheit: 77.00`.

Warning: use `double` for the temperature. If you use `int`, then `9 / 5` becomes 1 and every answer is wrong. Write `c * 9 / 5 + 32` with `c` as a `double`, or write `c * 9.0 / 5 + 32`.

2. Bill splitter. Ask for the total bill and the number of people. Print how much each person pays.

Example run: you type `100` and `3`, the screen shows `Each pays: 33.33`.

The total is a `double` (read it with `%lf`). The number of people is an `int` (read it with `%d`). Dividing a `double` by an `int` gives a `double`, so the decimals are kept.

3. Distance between two points. Ask for four numbers: x1, y1, x2, y2. Print the distance between the points (x1, y1) and (x2, y2).

The formula is the square root of $(x2 - x1)^2 + (y2 - y1)^2$. In C:

```
double d = sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
```

You need `#include <math.h>` at the top, and `-lm` at the end of the compile command on Linux and WSL.

Example run: you type `0 0 3 4`, the screen shows `Distance: 5.00`.

Before you say a program is done, check:

- It compiles with `-Wall` and no warnings.
- Every number it prints has a label in front of it.
- You ran it with at least two different inputs and the answers are right.
- The code is tidy: one instruction per line, lines inside `main` indented.

Challenge: money with exactly two digits after the point. You already know how: `%.2f`.

Slide 18 — Wrap-up

After this week you can:

- Write a C program, save it as a `.c` file, compile it, and run it.
- Use the terminal: `pwd`, `ls`, `cd`, `mkdir`, `gcc`, `./program`.
- Make variables (`int`, `double`, `char`) and put values in them.
- Print with `printf`, with the right hole (`%d`, `%.2f`, `%c`) and a label.
- Read a number with `scanf`, with `&` and with `%lf` for a `double`.
- Calculate with `+ - * / %`, and use `sqrt`, `pow`, `round`.

Before Tuesday.

1. **Do Knowledge check 1** on Blackboard, in the Session 1 folder. It is worth zero percent. You have three attempts and the best one counts. Do the practice questions too. They tell you what you did not understand while there is still time to ask.
2. **Write the three practice programs** from slide 17. Bring them on Tuesday, working or not.
3. **Compiler still not working?** Follow "Setting up a C compiler" on Blackboard. If you are stuck, post the exact error message in the *Questions and answers* forum. Nobody is behind because of a setup.

Next week: data types properly, why `7 / 2` is 3, and `if`, which lets a program make decisions.